# No Bird Sings Peirce: A Machine-Checked Separation of Classical from Constructive Implication

Working paper v0.1, Project secret3, August 11, 2026


## Abstract

In the Hilbert system over pure implication whose axioms are the combinators K and S and whose only rule is modus ponens, every derivation carries its proof term: the system is the propositions-as-types coin of the project's proof module, where a derivation of a proposition literally indexes the combinator that proves it. This paper closes the coin's classical border with a compiled theorem. A three-valued Heyting chain is defined inside the module, the two axioms are shown valid at every instantiation by exhaustive reflexivity, soundness is proved by one structural induction, and a single valuation then refutes both Peirce's law and double negation elimination. The results, peirceNoProof and dneNoProof, are universally quantified over proof terms: no term whatsoever, of any size, derives either law. Their constructive halves stand in contrast in the same module: identity, the syllogism, contraction, and double negation introduction are all derived, each with its explicit combinator. The separation is stated with its classical resolution alongside: Peirce's law is valid on every truth table and, by Griffin's correspondence, derivable the moment the term language admits a control operator; the theorem locates the border, it does not disparage the far side.


## 1. Introduction

The implicational fragment generated by K, S, and modus ponens is the oldest complete logic there is: Frege opened the Begriffsschrift with exactly these two axioms in 1879 [4], Schoenfinkel isolated the same two shapes as the atoms of computation in 1924 [5], and under the propositions-as-types reading the two discoveries are one object [7]. The fragment is constructive: it derives exactly the implicational truths of intuitionistic logic, and the classical surplus, Peirce's law (((A -> B) -> A) -> A) [1] and double negation elimination, lies outside it. That much is classical knowledge. The contribution of this paper is its promotion to a compiled theorem inside the project's total proof module: an internal semantics, a soundness induction, and two underivability results quantified over all proof terms, so that the claim is not that a search found no derivation but that no derivation exists.

The method is the standard finite countermodel, chosen for its mechanizability: the three-element Goedel chain [2], whose implication is Heyting's [3]. Sections 2 to 4 give the coin, the matrix, and the soundness proof; Section 5 the two refutations and the derivable contrast; Section 6 the scope, including Griffin's resolution [8]. Appendices reproduce the relevant module sections verbatim.

## 2. The coin

The project's module defines propositions and derivations as follows, with the derivation indexed by the very term that proves it (Appendix B):

```
data Ty = TV Nat | Arr Ty Ty

data Deriv : Ty -> Term -> Type where
```

```
  AxK : {a, b : Ty} -> Deriv (Arr a (Arr b a)) CK
  AxS : {a, b, c : Ty} ->
        Deriv (Arr (Arr a (Arr b c)) (Arr (Arr a b) (Arr a c))) CS
  MP  : {a, b : Ty} -> {f, x : Term} ->
        Deriv (Arr a b) f -> Deriv a x -> Deriv b (App f x)
```

On this coin the module already carries the positive inventory: iTyped (identity, proved by S K K), bTyped (the hypothetical syllogism, proved by S (K S) K), wTyped (contraction, proved by S S (S K)), and tTyped (from A follows that whatever A implies holds, which with absurdity as target is double negation introduction). The question this paper closes is what the coin cannot mint.

### 3. The matrix

Take three truth values on a chain, V0 below V1 below V2, with V2 designated, and Heyting's implication on the chain: x implies y is V2 when x is at most y, and y otherwise.

| timp | V0 | V1 | V2 |
|---|---|---|---|
| **V0** | V2 | V2 | V2 |
| **V1** | V0 | V2 | V2 |
| **V2** | V0 | V1 | V2 |

A valuation assigns a value to every type variable and extends to all propositions through the table. Because the chain is a Heyting algebra, both axioms are valid at every instantiation; in the module this is not cited but computed: kValid is seven reflexivity clauses, sValid is twenty seven, each case reduced by the checker to V2.

### 4. Soundness

Soundness is one structural induction: every derivable proposition takes the designated value under every valuation.

```
matSound : (v : Nat -> Val3) -> Deriv ty t -> teval v ty = V2
```

The axiom cases are kValid and sValid; the modus ponens case transports the premise's value into the implication and closes with a three-case lemma, mpValid, stating that a designated implication with designated antecedent has designated consequent. The whole proof is a page (Appendix C).

### 5. Two shut doors, one valuation

Let the refuting valuation read the first type variable as V1 and every other as V0. Peirce's law at concrete variables, ((a -> b) -> a) -> a, then computes through the table to V1: the antecedent a -> b falls to V0, the middle implication rises to V2, and V2 -> V1 is V1, short of designation. Double negation elimination, ((a -> f) -> f) -> a with the second variable as absurdity, computes to V1 under the same valuation by the same route. Soundness does the rest:

```
peirceNoProof : {t : Term} -> Deriv peirceTy t -> Void
dneNoProof    : {t : Term} -> Deriv dneTy t -> Void
```

The quantifier deserves its sentence. These theorems do not report a failed search over terms up to some size; they are universally quantified over the proof term, so that any derivation of either law, by any combinator of any size ever, yields an inhabitant of the empty type. And the contrast stands in the same module: the introduction half of double negation is derivable, tTyped, with S (K

(S I)) K as its explicit proof term and a six-move verified action. The door has two halves, and the coin opens exactly one of them.

### 6. Scope

Three boundaries, stated plainly. First, the system: this is underivability in the K, S Hilbert system over pure implication, which is the implicational fragment of intuitionistic and of minimal logic; nothing is asserted about richer signatures. Second, the classical side: Peirce's law is valid on every two-valued truth table, and classical mathematics is consistent, powerful, and the working language of most mathematicians; the theorem draws a border, not a verdict. Third, the resolution: by Griffin's correspondence [8], Peirce's law becomes derivable the moment the term language admits a control operator, so that the border is precisely priced: classical implication is constructive implication plus the ability of a computation to take back its word. Making that operator and its typing formal in this module is line L4 of the program's research agenda [11].

### Appendix A: Methods

All theorems live in the shared proof module Proofs.idr of the program, which stands at 1,523 lines after this paper's additions, is declared %default total, and contains no postulates and no holes; the single command idris2 —check Proofs.idr is the entire verification, and the entry point prints the theorem inventory. The twenty seven S-validity cases were generated programmatically and each is a single Refl for the checker. The companion papers reproduce earlier snapshots of the module and are updated at their own cadence.

### Appendix B: The coin (module part I)

```
--------------------------------------------------------------------
-- Tier 2, part I: the logic side (book chapters)
--------------------------------------------------------------------

||| simple types: propositions of the implicational fragment
public export
data Ty = TV Nat | Arr Ty Ty

||| Hilbert-style derivations: K and S as axioms, application as
||| modus ponens; a derivation types the very term that proves it
public export
data Deriv : Ty -> Term -> Type where
  AxK : {a, b : Ty} -> Deriv (Arr a (Arr b a)) CK
  AxS : {a, b, c : Ty} ->
        Deriv (Arr (Arr a (Arr b c)) (Arr (Arr a b) (Arr a c))) CS
  MP  : {a, b : Ty} -> {f, x : Term} ->
        Deriv (Arr a b) f -> Deriv a x -> Deriv b (App f x)

||| the two axioms, named for the chapters
kTyped : {a, b : Ty} -> Deriv (Arr a (Arr b a)) CK
kTyped = AxK

sTyped : {a, b, c : Ty} ->
  Deriv (Arr (Arr a (Arr b c)) (Arr (Arr a b) (Arr a c))) CS
sTyped = AxS

||| identity typed: A implies A, proved by S K K
iTyped : {a : Ty} -> Deriv (Arr a a) Proofs.iC
iTyped {a} =
```

```
  MP (MP (AxS {a = a} {b = Arr a a} {c = a})
         (AxK {a = a} {b = Arr a a}))
     (AxK {a = a} {b = a})

||| the syllogism typed: B proves (b -> c) -> ((a -> b) -> (a -> c))
bTyped : {a, b, c : Ty} ->
  Deriv (Arr (Arr b c) (Arr (Arr a b) (Arr a c))) Proofs.bC
bTyped {a} {b} {c} =
  MP (MP (AxS {a = Arr b c}
              {b = Arr a (Arr b c)}
              {c = Arr (Arr a b) (Arr a c)})
         (MP (AxK {a = Arr (Arr a (Arr b c)) (Arr (Arr a b) (Arr a c))}
                  {b = Arr b c})
             (AxS {a = a} {b = b} {c = c})))
     (AxK {a = Arr b c} {b = a})

||| the warbler: W x y steps to x y y, and its type is contraction
wC : Term
wC = App (App CS CS) (App CS CK)

wSteps : (x, y : Term) ->
  Steps (App (App Proofs.wC x) y) (App (App x y) y)
wSteps x y =
  Then (CtxL RedS) $ Then RedS $ Then (CtxR RedS) $ Then (CtxR RedK) Done

wTyped : {p, r : Ty} ->
  Deriv (Arr (Arr p (Arr p r)) (Arr p r)) Proofs.wC
wTyped {p} {r} =
  MP (MP (AxS {a = Arr p (Arr p r)} {b = Arr p p} {c = Arr p r})
         (AxS {a = p} {b = p} {c = r}))
     (MP (AxS {a = p} {b = Arr p r} {c = p})
         (AxK {a = p} {b = Arr p r}))

||| the forest theorem: every bird is fond of some bird; the fond
||| point of x is the mockingbird's answer to B x M, and the whole
||| solution is one mirror move followed by one composition
fondPoint : (x : Term) ->
  Steps (App Proofs.om (App (App Proofs.bC x) Proofs.om))
        (App x (App Proofs.om (App (App Proofs.bC x) Proofs.om)))
fondPoint x =
  stepsTrans (mirror (App (App Proofs.bC x) Proofs.om))
             (bSteps x Proofs.om (App (App Proofs.bC x) Proofs.om))
```

## Appendix C: The matrix and the separation (module part L)

```
------------------------------------------------------------------------
-- Tier 2, part L: no bird sings Peirce (L2)
------------------------------------------------------------------------

||| three truth values on a chain, V2 designated
public export
data Val3 = V0 | V1 | V2

||| Heyting implication on the chain: top if x is at most y, else y
timp : Val3 -> Val3 -> Val3
```

```
timp V0 _  = V2
timp V1 V0 = V0
timp V1 _  = V2
timp V2 V0 = V0
timp V2 V1 = V1
timp V2 V2 = V2

teval : (Nat -> Val3) -> Ty -> Val3
teval v (TV n)    = v n
teval v (Arr a b) = timp (teval v a) (teval v b)

||| the K axiom is valid at every instantiation
kValid : (x, y : Val3) -> timp x (timp y x) = V2
kValid V0 y  = Refl
kValid V1 V0 = Refl
kValid V1 V1 = Refl
kValid V1 V2 = Refl
kValid V2 V0 = Refl
kValid V2 V1 = Refl
kValid V2 V2 = Refl

||| the S axiom is valid at every instantiation
sValid : (x, y, z : Val3) ->
  timp (timp x (timp y z)) (timp (timp x y) (timp x z)) = V2
sValid V0 V0 V0 = Refl
sValid V0 V0 V1 = Refl
sValid V0 V0 V2 = Refl
sValid V0 V1 V0 = Refl
sValid V0 V1 V1 = Refl
sValid V0 V1 V2 = Refl
sValid V0 V2 V0 = Refl
sValid V0 V2 V1 = Refl
sValid V0 V2 V2 = Refl
sValid V1 V0 V0 = Refl
sValid V1 V0 V1 = Refl
sValid V1 V0 V2 = Refl
sValid V1 V1 V0 = Refl
sValid V1 V1 V1 = Refl
sValid V1 V1 V2 = Refl
sValid V1 V2 V0 = Refl
sValid V1 V2 V1 = Refl
sValid V1 V2 V2 = Refl
sValid V2 V0 V0 = Refl
sValid V2 V0 V1 = Refl
sValid V2 V0 V2 = Refl
sValid V2 V1 V0 = Refl
sValid V2 V1 V1 = Refl
sValid V2 V1 V2 = Refl
sValid V2 V2 V0 = Refl
sValid V2 V2 V1 = Refl
sValid V2 V2 V2 = Refl

mpValid : (y : Val3) -> timp V2 y = V2 -> y = V2
mpValid V0 Refl impossible
mpValid V1 Refl impossible
```

```
mpValid V2 e = Refl

||| soundness: every derivable proposition takes the designated value
||| under every valuation
matSound : (v : Nat -> Val3) -> Deriv ty t -> teval v ty = V2
matSound v (AxK {a} {b}) = kValid (teval v a) (teval v b)
matSound v (AxS {a} {b} {c}) =
  sValid (teval v a) (teval v b) (teval v c)
matSound v (MP {a} {b} df dx) =
  mpValid (teval v b)
    (replace {p = \z => timp z (teval v b) = V2}
             (matSound v dx) (matSound v df))

v1NotV2 : V1 = V2 -> Void
v1NotV2 Refl impossible

||| the refuting valuation: the first variable reads V1, all others V0
refuteVal : Nat -> Val3
refuteVal 0 = V1
refuteVal _ = V0

||| Peirce's law at concrete variables
peirceTy : Ty
peirceTy = Arr (Arr (Arr (TV 0) (TV 1)) (TV 0)) (TV 0)

||| no term whatsoever derives Peirce's law
peirceNoProof : {t : Term} -> Deriv Proofs.peirceTy t -> Void
peirceNoProof d = v1NotV2 (matSound Proofs.refuteVal d)

||| double negation elimination at concrete variables, with the
||| second variable as absurdity
dneTy : Ty
dneTy = Arr (Arr (Arr (TV 0) (TV 1)) (TV 1)) (TV 0)

||| no term whatsoever derives double negation elimination; its
||| introduction half is derivable, tTyped
dneNoProof : {t : Term} -> Deriv Proofs.dneTy t -> Void
dneNoProof d = v1NotV2 (matSound Proofs.refuteVal d)
```

## References


1. C. S. Peirce, ‘On the algebra of logic: a contribution to the philosophy of notation’, American Journal of Mathematics 7 (1885), 180-202.
2. K. Gödel, ‘Zum intuitionistischen Aussagenkalkül’, Anzeiger der Akademie der Wissenschaften in Wien 69 (1932), 65-66.
3. A. Heyting, ‘Die formalen Regeln der intuitionistischen Logik’, Sitzungsberichte der Preussischen Akademie der Wissenschaften (1930), 42-56.
4. G. Frege, *Begriffsschrift*, Halle, 1879.
5. M. Schoenfinkel, ‘Ueber die Bausteine der mathematischen Logik’, Mathematische Annalen 92 (1924), 305-316.
6. H. B. Curry and R. Feys, *Combinatory Logic*, Vol. I, North-Holland, 1958.
7. W. A. Howard, ‘The formulae-as-types notion of construction’ (1969), in *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, Academic Press, 1980.
8. T. Griffin, ‘A formulae-as-types notion of control’, *Proceedings of POPL 1990*, ACM, 1990.

9. E. Brady, ‘Idris 2: Quantitative Type Theory in Practice’, *Proceedings of ECOOP 2021*, LIPIcs 194, 2021.
10. Project secret3, ‘The Linearity Ladder: Where Erasure and Duplication Enter a Two-Combinator Arithmetic’, working paper v0.1, August 11, 2026.
11. Project secret3, ‘A Research Agenda for the Two Moves: Companion Papers to the Book’, research agenda v0.2, August 11, 2026.